CODES DES DIAGRAMMES UML

Exercices 1 à 4 — Programmation orientée objet en C++

Document contenant les codes PlantUML à copier dans un éditeur PlantUML pour générer les diagrammes.

Mode d'emploi : chaque bloc est indépendant. Copiez tout le code d'un bloc, puis collez-le dans un éditeur PlantUML (par exemple plantuml.com/).

EXERCICE 1 — Gestion d'une bibliothèque universitaire

1.1 — Diagramme de cas d'utilisation

```
@startuml
left to right direction
actor Etudiant
actor "Bibliothécaire" as Bibliothecaire

rectangle "Système de gestion de bibliothèque" {
  usecase "Consulter le catalogue" as UC1
  usecase "Emprunter un livre" as UC2
  usecase "Retourner un livre" as UC3
  usecase "Vérifier la disponibilité" as UC4
  usecase "Enregistrer un livre" as UC5
  usecase "Enregistrer un étudiant" as UC6
  usecase "Consulter les emprunts" as UC7
}

Etudiant --> UC1
Etudiant --> UC2
Etudiant --> UC3
UC2 .> UC4 : <<include>>
Bibliothecaire --> UC5
Bibliothecaire --> UC6
Bibliothecaire --> UC7
@enduml
```

1.2 — Diagramme de classes

```
@startuml
class Livre {
  - id : int
  - titre : string
  - auteur : string
  - anneePublication : int
  - disponible : bool
  + estDisponible() : bool
  + marquerEmprunte() : void
  + marquerDisponible() : void
}

class Etudiant {
  - matricule : string
  - nom : string
  - prenom : string
  - email : string
  + emprunter(l : Livre) : bool
  + retourner(l : Livre) : bool
  + afficherEmprunts() : void
}

class Emprunt {
  - dateEmprunt : string
  - dateRetour : string
  - actif : bool
  + enregistrerRetour() : void
  + estActif() : bool
}

Etudiant "1" -- "0..*" Emprunt : effectue
Livre "1" -- "0..*" Emprunt : concerne
note right of Emprunt
Un livre ne peut avoir
qu'un seul emprunt actif
à la fois.
end note
@enduml
```

EXERCICE 2 — Gestion des étudiants et des résultats académiques

2.1 — Diagramme de classes

```
@startuml
class Etudiant {
  - matricule : string
  - nom : string
  - prenom : string
  - niveau : string
  + calculerMoyenne() : double
  + validerUE(ue : UE) : bool
  + afficherReleve() : void
}

class Filiere {
  - code : string
  - libelle : string
  + ajouterUE(ue : UE) : void
  + afficherUE() : void
}

class UE {
  - code : string
  - libelle : string
  - credits : int
  + calculerMoyenne() : double
  + estValidee() : bool
}

class Matiere {
  - code : string
  - libelle : string
  - coefficient : double
  + definirCoefficient(c : double) : void
}

class Note {
  - valeur : double
  - session : string
  + modifierValeur(v : double) : void
  + estValide() : bool
}

Filiere "1" o-- "1..*" UE : regroupe
UE "1" *-- "1..*" Matiere : compose
Etudiant "0..*" -- "1" Filiere : appartient
Etudiant "1" -- "0..*" Note : obtient
Matiere "1" -- "0..*" Note : reçoit
note bottom of Note
Une note associe un étudiant
à une matière.
La moyenne d'une UE est
calculée à partir des notes
et coefficients des matières.
end note
@enduml
```

EXERCICE 3 — Gestion d’un personnel universitaire

3.1 — Diagramme de classes avec héritage et polymorphisme

```
@startuml
abstract class Employe {
  # matricule : string
  # nom : string
  # prenom : string
  # salaireBase : double
  + calculerSalaire() : double
  + afficherInformations() : void
}

class Enseignant {
  - indemnite : double
  + calculerSalaire() : double
}

class AgentAdministratif {
  - prime : double
  + calculerSalaire() : double
}

class Technicien {
  - primeTechnique : double
  + calculerSalaire() : double
}

Employe <|-- Enseignant
Employe <|-- AgentAdministratif
Employe <|-- Technicien

class ServicePersonnel {
  - employes : vector<Employe*>
  + ajouterEmploye(e : Employe*) : void
  + afficherTous() : void
  + calculerMasseSalariale() : double
}
ServicePersonnel "1" o-- "0..*" Employe : gère

note right of Employe
calculerSalaire() est virtuelle
et abstraite.
Chaque sous-classe redéfinit
la méthode selon ses primes.
end note
@enduml
```

EXERCICE 4 — Gestion d’un système de réservation de salles

4.1 — Diagramme de cas d’utilisation

```
@startuml
left to right direction
actor Enseignant
actor "Responsable des salles" as Responsable

rectangle "Système de réservation de salles" {
  usecase "Consulter les salles" as UC1
  usecase "Vérifier la disponibilité" as UC2
  usecase "Réserver une salle" as UC3
  usecase "Annuler une réservation" as UC4
  usecase "Consulter ses réservations" as UC5
  usecase "Gérer les salles" as UC6
}

Enseignant --> UC1
Enseignant --> UC3
Enseignant --> UC4
Enseignant --> UC5
UC3 .> UC2 : <<include>>
Responsable --> UC6
@enduml
```

4.2 — Diagramme de classes

```
@startuml
class Salle {
  - numero : string
  - capacite : int
  - categorie : string
  + estDisponible(date : string, debut : string, fin : string) : bool
}

class Enseignant {
  - matricule : string
  - nom : string
  - prenom : string
  + reserver(s : Salle, c : Creneau) : bool
  + annuler(r : Reservation) : bool
}

class Creneau {
  - date : string
  - heureDebut : string
  - heureFin : string
  + chevauche(c : Creneau) : bool
}

class Reservation {
  - id : int
  - statut : string
  + confirmer() : void
  + annuler() : void
}

class GestionReservations {
  - reservations : vector<Reservation>
  + verifierDisponibilite(s : Salle, c : Creneau) : bool
  + ajouterReservation(r : Reservation) : bool
  + supprimerReservation(id : int) : bool
}

Enseignant "1" -- "0..*" Reservation : effectue
Salle "1" -- "0..*" Reservation : accueille
Reservation "1" *-- "1" Creneau : concerne
GestionReservations "1" o-- "0..*" Reservation : gère

note right of GestionReservations
Refuser toute réservation
si une réservation confirmée
existe déjà pour la même salle
avec un créneau qui chevauche.
end note
@enduml
```

4.3 — Diagramme de séquence : réservation d’une salle

```
@startuml
actor Enseignant
participant "Interface" as I
```

```
participant "GestionReservations" as G
participant Salle
participant Reservation

Enseignant -> I : demanderReservation(salle, date, debut, fin)
I -> G : reserver(enseignant, salle, creneau)
G -> Salle : estDisponible(date, debut, fin)
Salle --> G : disponibilité

alt Salle disponible
  G -> Reservation : creer(enseignant, salle, creneau)
  Reservation --> G : réservation créée
  G -> G : enregistrer(reservation)
  G --> I : confirmation
  I --> Enseignant : afficherConfirmation()
else Salle déjà réservée
  G --> I : refus("Créneau indisponible")
  I --> Enseignant : afficherErreur()
end
@enduml
```

Remarques de modélisation

• Exercice 1 : la classe Emprunt permet de conserver l'historique des emprunts et retours. La contrainte métier impose au plus un emprunt actif par livre.

• Exercice 2 : la classe Note relie un étudiant à une matière. Les UE regroupent des matières ; les filières regroupent des UE. La règle de validation (par exemple moyenne ≥ 10/20) doit être confirmée par l'énoncé ou le règlement pédagogique.

• Exercice 3 : Employe est abstraite ; calculerSalaire() doit être déclarée virtuelle pure en C++ puis redéfinie dans chaque classe dérivée.

• Exercice 4 : le chevauchement de deux créneaux doit être testé. Deux créneaux se chevauchent si debut1 < fin2 et debut2 < fin1, pour une même date et une même salle.

Les diagrammes sont des propositions cohérentes avec les énoncés. Vous pouvez adapter les noms des attributs et les règles métier aux consignes de votre enseignant.